Julio Cesar Nogueira

Aula 03 - Arquitetura REST em Java

Revisão

O que fizemos no Laboratório?

- Play iniciou o servidor
- Spring Boot embute o Tomcat
- Aplicação é o servidor
- Erro 'Port in use'?
- Como matar processos na porta 8080

A Teoria das Camadas (O Fim da Célula Gigante)

O Problema do Jupyter Notebook

- Conexão única com o banco
- Limpeza de dados e modelagem
- Execução e plotagem em mesma célula
- Alto acoplamento entre etapas

Arquitetura de 3 Camadas

- Controller: recebe JSON, valida e devolve resposta
- Service: regra de negócio e cálculos
- Repository: interage com banco de dados

Hands-On Parte 1: A Camada Service

Tirando a lógica do Controller

- Abra o projeto no IntelliJ
- Crie pacote 'service'
- Adicione classe PredictionService

Código: PredictionService

```
package br.edu.faculdadedonaduzzi.dataservice.service;

import org.springframework.stereotype.Service;
import java.util.Random;

@Service // Essencial para registrar o Bean
public class PredictionService {
    public double calculateFraudRisk(String customerId,
                                     double transactionAmount) {
        System.out.println("Processando dados do cliente: " + customerId);
        double riskScore = (transactionAmount / 10000.0)
                         + new Random().nextDouble(0.5);
        return Math.min(riskScore, 1.0);
    }
}
```

Injeção de Dependência

Como o Controller acha o Service?

- new PredictionService() → instância manual
- @Autowired (ou via construtor) → injeção de dependência
- Padrão Singleton e economia de memória

Refatorando DataController

Injeção de Dependência e Query Params

```
package br.edu.donaduzzi.dataservice.controller;

import br.edu.donaduzzi.dataservice.service.PredictionService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1")
public class DataController {
    private final PredictionService predictionService;

    public DataController(PredictionService ps) {
        this.predictionService = ps;
    }

    @GetMapping("/predict")
    public String getPrediction(@RequestParam("id") String customerId,
                                @RequestParam("amount") double amount) {
        double score = predictionService.calculateFraudRisk(customerId, amount);
        return String.format("Risco de Fraude para %s é: %.2f%%", customerId,
                             score * 100);
    }
}
```

Do Texto para JSON

A API não devolve texto corrido

- Texto solto não é parseável
- DTO converte objetos em JSON
- Biblioteca Jackson serializa automaticamente

DTO: PredictionResponse

Definição e getters obrigatórios

```
package br.edu.donaduzzi.dataservice.dto;

public class PredictionResponse {
    private String customerId;
    private double riskScore;
    private String status;

    public PredictionResponse(String customerId, double riskScore) {
        this.customerId = customerId;
        this.riskScore = riskScore;
        this.status = riskScore > 0.7 ? "ALTO_RISCO" : "SEGURO";
    }

    public String getCustomerId() { return customerId; }
    public double getRiskScore() { return riskScore; }
    public String getStatus() { return status; }
}
```

Atualizando o Endpoint para JSON

Retornando PredictionResponse

```
// Alterar retorno de String para PredictionResponse
@GetMapping("/predict")
public PredictionResponse getPrediction(
    @RequestParam("id") String customerId,
    @RequestParam("amount") double amount) {
    double score =
predictionService.calculateFraudRisk(customerId, amount);
    return new PredictionResponse(customerId, score);
}
```

Atividade

Atividade1

- /api/v1/math/normalize
- Receber val1, val2, val3
- Lógica em nova MathService
- Retorno via MathResponse DTO

Atividade1

- Valores originais: Digamos que os números sejam 2, 6 e 10.
- Mínimo e Máximo: O menor número é 2 e o maior é 10.
- Cálculo para o primeiro valor (2): (2 - 2) / (10 - 2) = 0 / 8 = 0.0.
- Cálculo para o segundo valor (6): (6 - 2) / (10 - 2) = 4 / 8 = 0.5.
- Cálculo para o terceiro valor (10): (10 - 2) / (10 - 2) = 8 / 8 = 1.0.
- Resultado normalizado: 0.0, 0.5 e 1.0.

O que vem a seguir?

- Método POST em APIs
- Envio de corpos JSON complexos
- Documentação com Swagger